Assignment 2

Java has a built in string library which all are familiar with. This library provide the basic string manipulation methods which we are accustomed to. Here is a link to the Java 21 documentation: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/String.html

The purpose of this assignment is to introduce the concept of Junit testing. For this we will be using a custom version of the popular String library above. The entire library does not need to be implemented but sufficient amounts so that a viable Junit testing bed can be demonstrated. Below is the subset to be implemented.

Constructors

MyString()

Initializes a newly created String object so that it represents an empty character sequence.

MyString(char[] value)

Allocates a new String so that it represents the sequence of characters currently contained in the character array argument.

MyString(char[] value, int offset, int count)

Allocates a new String that contains characters from a subarray of the character array argument.

MyString(String original)

Initializes a newly created String object so that it represents the same sequence of characters as the argument; in other words, the newly created string is a copy of the argument string.

Methods:

char charAt(int index)

Returns the char value at the specified index.

Int compareTo(String anotherString)

Compares two strings lexicographically.

Int compareToIgnoreCase(String str)

Compares two strings lexicographically, ignoring case differences.

MyString concat(String str)

Concatenates the specified string to the end of this string.

Boolean contains(CharSequence s)

Returns true if and only if this string contains the specified sequence of char values.

Boolean endsWith(String suffix)

Tests if this string ends with the specified suffix.

Boolean equals(Object anObject)

Compares this string to the specified object.

Boolean equalsIgnoreCase(String anotherString)

Compares this String to another String, ignoring case considerations.

Int indexOf(int ch)

Returns the index within this string of the first occurrence of the specified character.

Int indexOf(int ch, int fromIndex)

Returns the index within this string of the first occurrence of the specified character, starting the search at the specified index.

Int indexOf(int ch, int beginIndex, int endIndex)

Returns the index within this string of the first occurrence of the specified character, starting the search at beginIndex and stopping before endIndex.

Int indexOf(String str)

Returns the index within this string of the first occurrence of the specified substring.

Int indexOf(String str, int fromIndex)

Returns the index within this string of the first occurrence of the specified substring, starting at the specified index.

Int indexOf(String str, int beginIndex, int endIndex)

Returns the index of the first occurrence of the specified substring within the specified index range of this string.

Boolean isEmpty()

Returns true if, and only if, length() is 0.

Int lastIndexOf(int ch)

Returns the index within this string of the last occurrence of the specified character.

Int lastIndexOf(int ch, int fromIndex)

Returns the index within this string of the last occurrence of the specified character, searching backward starting at the specified index.

Int lastIndexOf(String str)

Returns the index within this string of the last occurrence of the specified substring.

Int length()

Returns the length of this string.

MyString strip()

Returns a string whose value is this string, with all leading and trailing white space removed.

MyString substring(int beginIndex)

Returns a string that is a substring of this string.

MyString substring(int beginIndex, int endIndex)

Returns a string that is a substring of this string.

char[] toCharArray()

Converts this string to a new character array.

MyString trim()

Returns a string whose value is this string, with all leading and trailing space removed, where space is defined as any character whose codepoint is less than or equal to 'U+0020' (the space character).

Exceptions

IndexOutOfBoundsException - if the index argument is negative or not less than the length of this string.

NullPointerException - If any parameter object is null.

From the above Method description, create an interface called MyString, which will turn into MyString_Imp. From the method description note that all String types are MyString types, since we are writing our own. For example, MyString s = new MyString(char [] b); where b is some character array. It should also be beneficial to override the method String toString() so that output can be printed using standard system output.

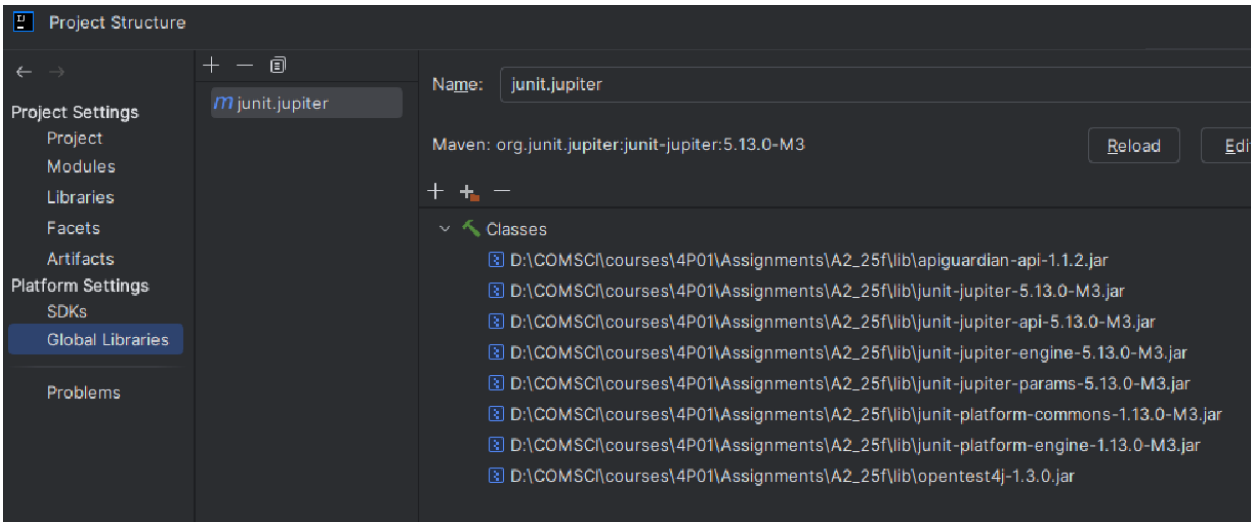
Junit testing is a testing framework for Test First Development. This is an educational assignment, in that you will research and learn the Junit test suit and apply it to the class definition above. The below link is a youtube tutorial for how to use Junit in intelliJ. But first we need to set up IntelliJ to provide Junit support.

Java Unit Testing with JUnit - Tutorial - How to Create And Use Unit Tests

https://www.youtube.com/watch?v=vZm0lHciFsQ

Junit support is built into IntelliJ, by way of a plug in. Start by creating a new project with the default "Hello World" code that IntelliJ provides. The above methods serve as an interface called MyString. You will then create MyString_Imp from the interface.

We are going to use Junit 5. From the project directory, open the modules (F4). You should get a blank version of the following screen.



Go to the Global Libraries: and in the 2nd column hit the "+" and go to the maven submenu. Follow the screens and search for org.junit.jupiter, install the one as shown above. Note, there could be a newer version but this one has worked in the past.

Lets assume you now have MyString and MyString_Imp, where MyString_Imp is effectively empty, you have written no code. Write the first 2 constructors as listed, and a simple method, say length. From the video, (about 2 min in) create a test class.

Your job will be to populate the test class with test cases using Junit 5. For every chunk of code you write in MyString_Imp, you must provide extensive tests to ensure it is correct. These include exception verification, parameter verification, and functionality verification. The video does a good job getting you started.

When you are finished you should be able to run the test and verify that all cases have been covered, and that your class is sound.

What this assignment is not. Turning in a working class without any test suit will earn you a big fat zero. It is not about writing a java class, it is about learning Junit testing. It also means your class has to be correct. Thus, marks are assigned for completeness and thoroughness of the testing and the construction of the test environment.

Each method and constructor should have sufficient tests to ensure that all cases are taken care of. E.g. if we perform a substring with a begin and end index, then it is required that these be within the bounds of the MyString. If end <= to the begin, then end = begin and the result should be a MyString of 1 character in length. Nowhere in your MyString_Imp are you to use a String except for toString().

Marking will be based on the quality of the submission which consists of robust class testing.

Please start early.

Submission

- To be submitted via Brightspace
- As part of your submission you are required to provide a complete copy of your source code for the classes you use/create. These will be MyString, MyString_Imp, MyString_ImpTest, any user exceptions that you create. This can be a pdf.
- A zipped up IntelliJ project. Assignments will be awarded a mark of zero if the project is not submitted, and viable.
- Only IntelliJ projects are accepted. We can not support every conceivable platform, thus, everyone will use IntelliJ.

End